Byzantine Fault-Tolerant Payments: Consensus-free, PBFT and Blockchain Replication

António Mendes*
an.mendes@campus.fct.unl.pt
MEI, DI, FCT, UNL
Caparica, Portugal

Francisco Almeida†
fs.almeida@campus.fct.unl.pt
MEI, DI, FCT, UNL
Caparica, Portugal

Bernardo Gracioso‡
b.gracioso@campus.fct.unl.pt
MEI, DI, FCT, UNL
Caparica, Portugal

ABSTRACT

We design, implement and evaluate a replicated payment service that tolerates Byzantine faults in both replicas and clients: a (very) simplified version of the systems that manage account balances in a bank or in a cryptocurrency. The service must guarantee that money is never created or destroyed, that only the owners of an account can spend its funds, and that the same funds are never spent twice. On top of a fixed application layer and the Babel 2 framework, we build three replication solutions with different guarantees: consensus-free payments over Byzantine Reliable Broadcast, which imposes no total order on transfers; state machine replication over PBFT, which yields a consistent total order; and a Proof-of-Work blockchain, where blocks of transfers establish a tamper-evident and eventually consistent order. We argue the correctness of each solution in a partially synchronous system with $n \geq 3f + 1$ replicas and aim to compare them in terms of throughput, latency and consistency when accounts are shared by several owners. We evaluate all three alternatives in a local Docker environment. For Blockchain, confirmation latency and throughput vary with proof-of-work difficulty and confirmation depth. For BRB, read-only throughput is high, but shared-account contention can leave correct replicas with divergent ledgers because broadcasts have no global order. PBFT provides ordered execution in the four-replica runs, including recovery from a one-shot equivocation and a primary crash. After increasing retained history, all four-replica client-load runs converged. The initial targeted seven-replica equivocation runs exposed a lagging correct replica; after adding quorum-certified tail catch-up, all correct replicas converged in four repeated runs. Blockchain safety experiments also show that irreversible payment execution can conflict with a deep reorganization. These results characterize the implementations and setup; they are not security guarantees for arbitrary schedules or mining-power distributions. PBFT results and the limits of the current comparison are discussed below.

*Student number 73530. Blockchain development
†Student number 75116. BRB and State Machine
‡Student number 75486. PBFT and State Machine

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
RDS 26/27, 2026, Faculdade de Ciências e Tecnologia, NOVA University of Lisbon, Portugal
© 2026 Association for Computing Machinery.

ACM Reference Format:
António Mendes, Francisco Almeida, and Bernardo Gracioso. 2026. Byzantine Fault-Tolerant Payments: Consensus-free, PBFT and Blockchain Replication. In *Reliable Distributed Systems 2026/27 – first project, Faculdade de Ciências e Tecnologia, NOVA University of Lisbon, Portugal*. ACM, New York, NY, USA, 13 pages.

1 INTRODUCTION

Systems that manage account balances, such as those of a bank or of a cryptocurrency, must remain available and trustworthy even when some of their components fail or are compromised. Replicating the service tolerates crashes, but when replicas (and clients) can behave arbitrarily, i.e. under the Byzantine fault model, the replicated service must still ensure that money is never created or destroyed, that only the owner of an account can spend its funds, and that the same funds are never spent twice (*double spending*).

This report documents a replicated payment service that lets clients transfer funds from an account they own to any other account, read the balance of an account, and read the history of transfers (outgoing and incoming) of an account.

It is well known that state machine replication requires consensus to order operations. However, a payment system where each account has a *single owner* does not need consensus: it is enough that all transfers from the same account are executed in the order defined by its owner, and transfers from different accounts commute (in the absence of double-spending attempts) [6]. This result inspired practical systems [4], but stops being true as soon as accounts can be shared by several owners. A central goal of our work is to observe, explain and measure the consequences of this observation.

To this end, we implemented three replication solutions underneath the same application logic, each with distinct guarantees:

- **Consensus-free payments**, based only on Byzantine Reliable Broadcast [1, 2] between replicas, without a global (total) order on transfers;
- **PBFT-based payments**, where state machine replication over Practical Byzantine Fault Tolerance [3] provides a consistent total order;
- **Blockchain-based payments**, a replicated blockchain heavily inspired by Bitcoin [8], where blocks of transfers establish a tamper-evident and eventually consistent order.

All of them are implemented in Java using Babel 2, the evolution of the Babel framework [5].

The evaluation compares all three alternatives under the same application. This draft now includes local measurements for Blockchain,

BRB and PBFT. The workload and protocol parameters are not identical in every scenario, so the final matched three-way comparison remains open.

The remainder of the paper is organized as follows. Section 2 introduces the payment service and the building blocks we rely on; Section 3 states the system model and the threats we consider; Section 4 describes the architecture and the three solutions; Section 5 argues their correctness; Section 6 presents the experimental evaluation; and Section 7 concludes.

2 BACKGROUND

2.1 The Payment Service

The service manages a fixed set of *accounts*, each with a name, a balance and a set of *owners*. Owners are clients, identified by a public key known to all replicas. In the initial state of the system (the *genesis*), every client owns an account with its own name, a few *shared accounts* are owned by several clients, and all accounts start with the same balance.

A *transfer* is the operation replicated by the system. It is composed of $\langle from, to, amount, seq, signer \rangle$ and a signature of the signer over these fields. A transfer is *valid* if both accounts exist and are different, the amount is positive, the signer is one of the owners of account $from$, and the signature is correct. Each account has its own sequence of transfers: the transfer of account A with sequence number s can only be executed after the transfers of A with sequence numbers $0, 1, \ldots, s-1$. Two different transfers from the same account with the same sequence number are *conflicting*: at most one of them can ever be executed.

Each replica keeps its state in a deterministic *ledger*. When a valid transfer is handed to the ledger, it is either (i) *executed*, if it is the next transfer of its account and the account has enough funds (which may trigger the execution of other pending transfers); (ii) kept *pending*, if previous transfers of the account were not executed yet or the account lacks funds; or (iii) *rejected as a conflict*, if a different transfer of the same account with the same sequence number was handed to the ledger first. If every account has a single (correct) owner, the final ledger state does not depend on the order in which transfers of different accounts are handed to it; with shared accounts, two owners may concurrently issue conflicting transfers, and replicas that see them in different orders diverge.

Clients. Since replicas may be Byzantine, a client cannot trust the reply of a single replica (Figure 1): it sends each request to all replicas and accepts a result only after receiving $f+1$ matching replies from different replicas, which ensures that at least one correct replica vouches for it. Replicas reply to a transfer once its outcome is known (executed, invalid, or conflicting), while balance and history requests are answered immediately from the replica's local state. To issue a transfer, a client must know the next sequence number of the account: for its own account it simply keeps track of it, while for a shared account it must first read it with a balance request, and other owners may be doing the same concurrently.

2.2 Building Blocks

Byzantine Reliable Broadcast (BRB).. BRB [1, 2] lets a process request the broadcast of a message (`brbBroadcast(m)`) and notifies

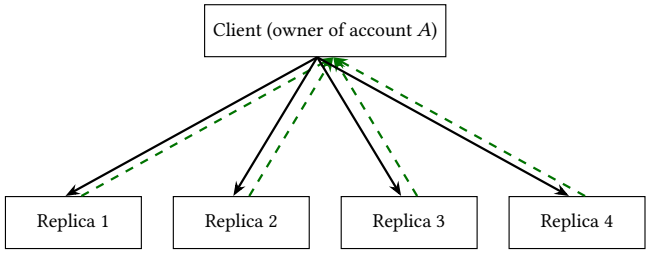


Figure 1: Interaction between a client and the replicas. Solid arrows: the signed request is sent to all replicas. Dashed arrows: replies; the result is accepted after $f+1$ matching ones.

the delivery of messages (`brbDeliver(p, m)`) such that, even if the sender is Byzantine, all correct processes deliver the same message for a given broadcast instance (*consistency*), if one correct process delivers a message then all correct processes eventually do (*totality*), and a message broadcast by a correct process is eventually delivered by all correct processes (*validity*). BRB does *not* order messages from different instances.

PBFT.. Practical Byzantine Fault Tolerance [3] implements state machine replication with $n \geq 3f+1$ replicas, f of which may be Byzantine. In each view, a primary assigns sequence numbers to (batches of) requests through the pre-prepare, prepare and commit phases; checkpoints bound the log; and a view change replaces a primary suspected to be faulty. Safety holds in asynchronous systems, and liveness requires partial synchrony. In this project the membership is static, so a view change only replaces the primary.

Proof-of-Work blockchains. In Bitcoin-style systems [8], transfers are grouped into blocks; each block references the hash of the previous block, forming a chain, and includes a proof-of-work (a nonce such that the hash of the block has a given number of leading zeros). Participants adopt the longest valid chain. A block is considered stable once it has k confirmations, so consistency is eventual and, in general, probabilistic.

2.3 Babel 2

We use Babel 2, the evolution of the Babel framework [5] developed in the NOVA LINCS laboratory. Protocols are event-driven state machines written in Java, declaring their handlers through annotations; messages are Java records whose serializers are generated automatically; and requests and indications between protocols of the same process are events. Inter-process communication relies on the Netty framework, which the framework hides from the programmer. Channels authenticate the identity of the other side when a connection is established, and a message is delivered to *every* protocol that handles its type, independently of the channel where it was received.

3 SYSTEM MODEL AND THREATS

We consider a partially synchronous system under the Byzantine fault model, with a static set of $n \geq 3f+1$ replicas, of which at most f can be Byzantine. Any number of clients can be Byzantine,

Table 1: Threats and protocol treatment.

Threat	Handling / mitigation
Unauthorized, altered or replayed transfers	The payment service verifies ownership and signatures; the ledger enforces sequence numbers and duplicate-ID protection.
Conflicting spends from one account	The ledger accepts at most one transfer for an account sequence. Replicas need a common order to make the same choice. BRB alone gives no total order.
Byzantine replica equivocation, omission or selective relay	BRB thresholds protect agreement per instance, not ordering across instances. PBFT uses quorum certificates and view changes. Proof of work gives probabilistic chain stability.
Invalid or conflicting PBFT proposals	Correct replicas validate requests and certificates before execution; view changes are intended to restore progress after a faulty primary. PBFT implementation results are reported separately.
Invalid blocks, withholding and chain rewrites	Blockchain validation rejects malformed blocks and transfers. Longest-chain selection and k confirmations do not prevent every deep reorganization; executed payments are irreversible.
Nonmembers, crashes or link loss	Handlers check authenticated membership. Liveness depends on eventual synchrony and enough responsive replicas; finite disruption runs test selected schedules only.

but a correct client never discloses its private key. Processes communicate through authenticated perfect point-to-point links: the channels provided by Babel 2 verify the identity of the other side when a connection is established, so the sender of every message is authenticated. Whenever a process must convince a third party that some process said something (e.g., to build certificates in PBFT), messages must be explicitly signed. Cryptographic primitives cannot be broken.

Table 1 lists the attacks we considered and how each one is handled (or mitigated) by our solutions.

4 ARCHITECTURE AND IMPLEMENTATION

4.1 Layering

Each replica is composed of three layers (Figure 2): the *payment service* (application logic, provided); a *replication protocol* (simple replication, state machine replication, or blockchain); and a *dissemination/agreement protocol* (Byzantine Reliable Broadcast or PBFT). The BRB protocol is used by two of the alternatives, so it is implemented once and reused. The interfaces between layers are:

- payment service ↔ replication protocol (all variants): request `SubmitTransaction(tx)` and indication `ExecuteTransaction(tx)`, which hands one transfer at a time to the ledger in the required execution order;
- replication protocol ↔ BRB: request `brbBroadcast(m)` and indication `brbDeliver(p, m)`;
- state machine replication ↔ PBFT: requests `Propose(batch)` and `SuspectPrimary(view)`, and indications `Decided(sqn, batch)` (in sequence number order, without gaps) and `ViewChanged(view, primary)`.

4.2 Application Layer

The application layer (payment service and ledger) is provided and shared by the three solutions. When a replica receives a valid transfer from a client, it submits it to the replication protocol, and hands it to the ledger only when the replication protocol indicates that it should be executed. As clients send their requests to all replicas, the same transfer is typically submitted by several replicas, and a Byzantine replica or client might submit anything, including old or invalid transfers. The ledger ignores duplicates, but the replication protocols should avoid wasting resources disseminating or ordering the same transfer several times. The application submits each transfer only *once*, so it is the replication protocol's responsibility to ensure that every submitted transfer is eventually executed (e.g., even across view changes). Periodically, the application logs a digest of its state (balances, next sequence numbers and executed transfers of every account): correct replicas that executed the same transfers have the same digest, regardless of the order, which we use to check whether they converged.

4.3 Byzantine Reliable Broadcast

For a broadcast instance, the origin sends SEND to the replicas. A replica accepts at most one ECHO value from each sender and sends ECHO for the first valid SEND it accepts. It sends READY after receiving $\lfloor (n + f)/2 \rfloor + 1$ matching ECHOs, or relays READY after receiving $f + 1$ matching READY messages. Delivery occurs after $2f + 1$ matching READY messages and is recorded once per instance. The thresholds use the configured static membership; messages from non-members are not counted.

4.4 Consensus-free Payments

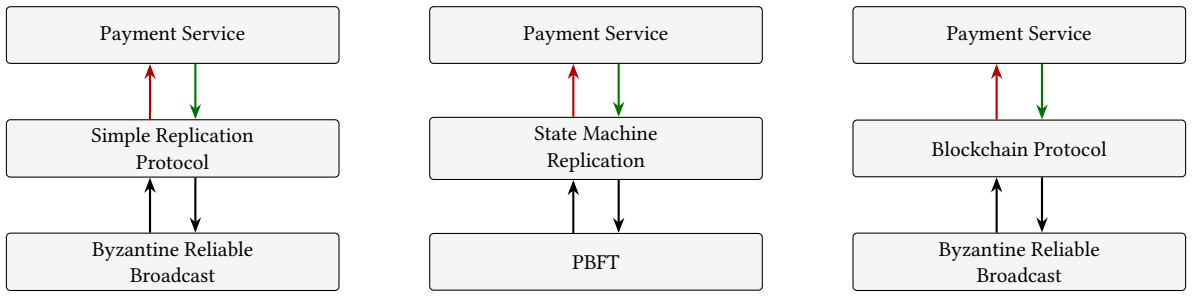
In this variant the replication protocol has almost no logic of its own: transfers submitted by the application are handed to BRB, and the replication protocol hands them to the application as soon as they are delivered. Correct replicas eventually deliver the same set of transfers, but no global order is guaranteed.

The protocol instance is identified by both origin and instance UUID, so equal UUIDs from different origins are distinct instances. Per-sender vote tracking prevents duplicate messages from increasing a threshold. Quorum intersection under $n \geq 3f + 1$ prevents two correct replicas from delivering different values for one instance. BRB does not order different instances; consequently, it does not resolve concurrent conflicting transfers to a shared account in one global order.

4.5 State Machine Replication over PBFT

In this variant the replication protocol is a state machine replication layer on top of PBFT. The primary groups transfers into batches and proposes them to PBFT, which establishes a consistent total order; decided batches are executed in sequence number order. When the primary is replaced, pending transfers must not be lost, but eventually ordered and executed: replicas forward transfers to the primary and suspect it (`SuspectPrimary`) when transfers are not ordered in due time. Every PBFT message (pre-prepare, prepare, commit, checkpoint, view-change and new-view) is digitally signed by its sender; the MAC/authenticator optimizations of the original paper are not used.

Batch size, flush timeout, view-change timers and checkpoint garbage collection are implementation parameters that must be reported alongside the PBFT measurements.



(a) Consensus-free payments

(b) State machine replication using PBFT

(c) Blockchain (Proof-of-Work)

Figure 2: Architecture of a replica and its three variants. Between the payment service and the replication protocol, the red upward arrow is the `ExecuteTransaction` indication and the green downward arrow is the `SubmitTransaction` request; the black arrows are the requests and indications of the dissemination/agreement layer.

4.6 Blockchain

In this variant the replication protocol groups transfers into blocks. Each block references the hash of the previous block and includes a proof-of-work: a nonce such that the hash of the block has a configurable number of leading zeros. Replicas act as miners and disseminate the blocks they mine through BRB. Each replica validates the blocks it receives, manages its local view of the chain, and adopts the longest valid chain. Once a block is k blocks deep (k is a parameter), its transfers are considered stable and are handed to the application, which never rolls a transfer back; hence a transfer is executed only after k blocks were appended after the block that includes it.

A block contains a parent hash, height, an ordered list of signed transfers and a nonce. Its hash is computed from a versioned canonical encoding of these fields; proof of work requires the configured number of leading zero bits in the SHA-256 hash. The bounded transaction pool filters invalid and already-finalized transfers and supplies at most the configured block size to an asynchronous miner. Replicas disseminate mined blocks using BRB, validate block size, proof of work, parent, height, transfer validity and duplicate transfer IDs, and retain bounded orphans until their parent is received. Fork choice selects the greatest height, breaking equal-height ties by the lexicographically smaller block hash. The measured configuration uses block size 100, difficulty 16 bits and $k = 2$ unless a sweep varies that parameter. A selected-chain block is handed to the application after k descendants; once handed off, its effects are irreversible.

5 CORRECTNESS

For each alternative, we argue why correct replicas never execute an invalid transfer, why money is never created or destroyed, and why there is no double spending (or, when that is not the case, under which conditions it can happen and why), in a partially synchronous system under the Byzantine fault model.

5.1 Common Ground

All variants rely on the payment service to verify account existence, distinct source and destination, positive amount, signer ownership and transfer signature. The ledger applies a transfer only at the next sequence number for its source account and only when sufficient funds are available; it rejects a competing transfer at an already occupied sequence number and debits and credits the same amount. Thus every individual ledger transition preserves total supply and cannot create a negative balance. These local properties do not by themselves ensure that correct replicas choose the same conflicting transfer: that property depends on the replication protocol.

5.2 Consensus-free Payments

BRB agreement ensures that correct replicas deliver the same value for a given broadcast instance. With a correct single owner issuing a sequence of transfers, each valid transfer is disseminated and the ledger enforces the account sequence, so a later transfer cannot bypass an earlier one. Under eventual synchrony, reliable links and at most f faulty replicas, the BRB thresholds and $n \geq 3f + 1$ support delivery from a correct origin. This is not a total order across broadcast instances. Concurrent owners of a shared account, or a Byzantine owner issuing conflicting signed transfers, can therefore lead replicas to process conflicts in different orders; the ledger prevents local double execution but the variant does not guarantee global convergence for these cases.

5.3 PBFT-based Payments

PBFT is intended to provide a common sequence of decided batches. With $n \geq 3f + 1$, prepare and commit quorums intersect in correct replicas; signed protocol messages let replicas verify the evidence supporting a decision. A correct implementation must validate requests before execution and carry prepared values through view changes, so a faulty primary cannot make correct replicas decide conflicting values for one sequence number. Under partial synchrony, a view change away from an unresponsive primary is the mechanism for restoring progress. These arguments describe the intended PBFT design; implementation-specific validation and experimental evidence must be checked against the completed protocol.

5.4 Blockchain-based Payments

Replicas add a block to the connected block tree only after checking its proof of work, parent hash, height, size, transfer validity and duplicate transfer IDs. They select the longest valid chain, but do not exchange a consensus certificate for the selected tip. A k-deep block is therefore a probabilistic stability heuristic whose security depends on assumptions about relative mining power and network propagation; it is not an absolute finality guarantee. A sufficiently long competing chain can replace the selected chain after a transfer was handed to the payment service. Since the service has no rollback interface, it cannot undo that execution. If conflicting transfers from competing branches have already been handed off at different replicas, their ledgers can diverge even though each local transition preserves supply. Transfers from abandoned blocks that were not yet handed off are not executed by virtue of that branch.

6 EXPERIMENTAL EVALUATION

6.1 Setup

Experiments used the supplied Docker setup on one local host, not on the DI and NOVA LINCS cluster. The host was an Apple M5 running macOS 27.0.1; Docker 29.9.2 exposed a Linux 7.0.14 VM with 10 CPUs and about 7.9 GiB of memory. Experiment containers used Eclipse Temurin OpenJDK 25.0.5. Blockchain and BRB replicas were normally limited to one CPU and 1 GiB each; the seven- replica Blockchain follow-up used 700 MiB per replica. PBFT containers had no explicit CPU or memory limits, so they shared the Docker VM's available resources. Client containers had no explicit resource limit. Runs used four replicas ($n = 4$, $f = 1$), except the replica-count sweeps, which also used seven ($n = 7$, $f = 2$). The f value is the tolerated fault bound; performance runs injected no faults unless a fault scenario is named. The retained run manifests record client arguments, protocol parameters, network delay, fault events and container resource limits. Scripts, environment details and raw results are retained locally under the Git-ignored `benchmarks/` directory.

The supplied client generates closed-loop workloads: each logical client executes one operation at a time. Runs have preparation, measurement and cooldown periods. Client output includes completed, failed and timed-out operations, throughput, average and percentile latency, per-second completions and divergent replies observed before quorum acceptance. Audited runs record final digests, balances, sequence numbers, executed and pending transfer IDs, and total-supply checks. Completed-operation throughput must not be confused with successfully executed-transfer throughput. Runs were collected sequentially by scenario rather than in a randomized order, and all clients and replicas shared one Docker host; host load and time-dependent effects therefore remain possible confounders. With three runs at most principal settings, means and standard deviations are descriptive summaries, not statistical significance tests.

6.2 Blockchain evaluation

6.2.1 Performance: P1–P5. Table 3 defines the six Blockchain performance and disruption scenarios. P6 outcomes are reported with the safety observations below because its primary checks are recovery and replica-state convergence.

Table 2: Client baseline and parameter sweeps.

Parameter	Value
Logical clients	5 baseline; 1, 5, 10, 20, 50 sweep
Transfer mix	80% baseline; 100% for fork/shared-account runs
BRB transfer/balance mix	0/100, 50/50, and 100/0%
History / shared mix	0% baseline; Blockchain shared runs use 100% shared; BRB sweep uses 0/50/100%
Reply quorum / timeout	$f + 1$ / 5000 ms
Prepare / run / cooldown	3000 / 30000 / 5000 ms baseline
Added link delay	Blockchain: 1 ms; BRB: 0 ms
Replicas / faults	4 / 1 baseline; 7 / 2 in replica sweep
Block size / difficulty / k	100 / 16 bits / 2 baseline
Sweeps	$k = 0, 1, 4$; block size 1, 10, 50, 100; difficulty 16, 18, 20
Repetitions	3 per principal setting; selected follow-ups have 6

Table 4 reports the full client-side performance sweeps. At four replicas, increasing the closed-loop client population from 1 to 50 raised successful client-response throughput from 12.67 ± 0.72 to 73.20 ± 15.10 responses/s, while the mean of the per-run medians rose from 69.3 to 552.5 ms and mean p95 rose from 160 to 1603 ms. The larger client population therefore raised aggregate throughput while each request took longer; the 50-client point also had high run-to-run variability. These clients issue work as soon as the preceding request completes, so this is a concurrency sweep rather than a fixed arrival-rate or saturation-capacity test.

Increasing proof-of-work difficulty from 16 to 20 bits reduced the observed successful-response rate from 40.38 ± 0.69 to 2.94 ± 0.67 responses/s and increased mean p50 latency from 105.8 to 1428.1 ms. This is consistent with the additional mining work in this CPU-limited local setup; it is not a measurement of security against a specified real-world hash-power distribution. Increasing k from 0 to 4 reduced throughput from 55.63 ± 2.59 to 32.39 ± 2.26 responses/s and increased p50 from 68.9 to 139.9 ms. At $k = 0$, a transfer is reported without waiting for a successor block, so this is a performance endpoint with no confirmation- depth protection.

The block-capacity sweep is not monotonic: the highest mean observed rate was at capacity 10, and capacity 100 had notably larger run-to-run variation. Capacity is an upper bound, not a target fill level; these data do not establish that increasing the block limit causes lower throughput. The seven-replica runs achieved 31.13 ± 2.77 responses/s versus 39.85 ± 0.43 at four replicas, but the comparison also changes f, adds miners, lowers the per-replica memory limit from 1 GiB to 700 MiB, and has six rather than three runs at seven replicas. Treat this as a local scaling observation, not an isolated estimate of communication cost. The mean p50 also rose from 109.9 ± 2.6 ms at four replicas to 138.9 ± 12.9 ms at seven. Across resource-instrumented Blockchain sweeps, replica

Table 3: Blockchain P1–P6 scenario definitions and primary measurements.

ID	Scenario	Variation and primary measurements
P1	Client load	Logical clients: 1, 5, 10, 20, 50; successful transfer responses/s, p50/p95 latency and failures/timeouts.
P2	Confirmation depth	$k = 0, 1, 2, 4$; throughput and client response latency as confirmation depth changes.
P3	Block capacity	Maximum transfers per block: 1, 10, 50, 100; response throughput and latency.
P4	Proof-of-work difficulty	16, 18, 20 leading-zero bits; throughput, latency and failed/timed-out requests.
P5	Replica-count scaling	Four versus seven replicas under the recorded client and resource settings; throughput and latency.
P6	Timed disruption and recovery	Replica crash and 10-second node4 egress-loss schedules; client outcomes, supply conservation and final digest convergence.

CPU averaged about 99–101% of the one-vCPU limit, making CPU saturation a material constraint. These measurements characterize one local Docker host and are not directly comparable to the course cluster. The baseline Blockchain runs added 1 ms of link delay, while BRB runs used 0 ms. Their raw performance values should therefore not be ranked as if network conditions matched; a final cross-protocol comparison must align link delay, workload, resource limits and repetition count.

Values are means and standard deviations across repeated runs. The p50 and p95 columns are means of the corresponding per-run percentiles, not percentiles pooled across requests. Principal settings have three runs; difficulty, block capacities 10/50/100 and the seven-replica follow-up have six retained runs. The run count is shown in Table 4. Figure 4 visualizes the concurrency and block-capacity trends with run-to-run variation.

6.2.2 Safety: S1–S6. Table 5 defines the Blockchain safety scenarios referenced in the results below.

S1–S3: Validation, delivery order and pre-finality forks. Deterministic Blockchain tests cover invalid proof of work, malformed parent, incorrect height, oversized blocks, invalid transfer signatures, duplicate transfer IDs, duplicate delivery and out-of-order orphan attachment. Rejected input does not poison a later valid block. A separate two-replica test gives the replicas the same competing valid blocks in opposite orders and confirms they finalize only the deterministically selected branch. These are controlled correctness tests, not randomized performance trials.

S4: Fork after a transfer is executed. The deterministic S4 counterexample uses two protocol instances backed by separate ledgers. One ledger executes Alice's sequence-0 transfer to Bob after it reaches $k = 1$; the other executes a conflicting sequence-0 transfer to Carol on a longer branch. Once the first instance learns the longer branch, its chain view changes, but its already-applied payment effect cannot be undone. The conflicting transfer is rejected by that ledger, while the second ledger has already applied it; their executed-ID sets and digests differ even though both preserve supply. This deterministic test demonstrates the no-rollback limitation; it does not estimate the probability of a deep reorganization under a particular mining-power distribution. In separate 30-second process-level longest-chain runs using four replicas, five independent clients, difficulty 16, $k = 2$, 0 ms added delay and no replica resource caps, logs recorded 2,680–2,765 chain-reorganization events per run, yet all correct replicas converged at the final audit in all three runs. Those finite runs did not reproduce the deterministic post-finality counterexample and are not evidence that it cannot occur.

S5: Byzantine miner behavior. Process-level Blockchain Byzantine tests used four replicas with one faulty replica and three repetitions per profile. These 15-second fault runs used five closed-loop clients, difficulty 16, $k = 2$, no added link delay, and no replica CPU or memory cgroup limits. Correct replicas rejected all 920–1,057 injected invalid blocks in each invalid-block run. The withholding profile logged 1,656–1,868 withheld blocks per run; the selectively sending profile logged 886–1,012 selective-send events per run. Correct replicas converged and conserved supply in all repetitions. These runs demonstrate the observed outcomes for these schedules, not liveness under every Byzantine strategy.

P6: Timed disruption and recovery. Three Blockchain crash runs stopped one replica 10 seconds after client startup. The three surviving replicas converged in every run and conserved supply. No transfer timed out; one balance request timed out in one repetition.

We also repeated the Blockchain network-disruption experiment after verifying the fault hook. In each of three 30-second runs, the harness applied 100% packet loss to node4's outbound interface from 10 to 20 seconds, then removed the loss rule. This is one-way egress loss, not a bidirectional network partition. The runs used four replicas, one fault, difficulty 16, block capacity 100, $k = 2$, five closed-loop clients, 1 ms added link delay, and 1-vCPU/1-GiB replica limits. All three runs completed client transfer requests without reported failures or timeouts, and every audited replica conserved supply. However, only one run ended with matching full ledger snapshots: after the stabilization wait, node4 matched the other replicas in run 1, while in runs 2 and 3 nodes 1–3 agreed with each other and node4 had a different, behind state. This shows that the harness's state-stability condition can be reached while replicas still disagree. The earlier unverified partition attempts are excluded.

To determine whether the short-run lag persisted during continued operation, we repeated the same disruption three times with a 90-second client workload instead of 30 seconds. In all three longer runs, the replicas had matching final snapshots after the harness stabilization period, and the clients completed 5,618–5,961 transfers without failures or timeouts. This is consistent with a lagging replica catching up while mining and client traffic continue. Because the follow-up also extends the active workload, it does not

Table 4: Blockchain client-side performance. Values are mean ± sample standard deviation. Throughput counts successful client transfer responses per second; p50 and p95 values are means of per-run percentiles. The final column gives total failed and timed-out transfers across the runs.

Sweep / setting	Runs	Responses/s	p50 (ms)	p95 (ms)	Failed / timed out
Clients: 1	3	12.67 ± 0.72	69.3 ± 4.3	160.3 ± 11.4	0 / 0
Clients: 5 (4 replicas)	3	39.85 ± 0.43	109.9 ± 2.6	250.4 ± 13.8	0 / 0
Clients: 10	3	56.63 ± 3.75	141.3 ± 18.9	419.9 ± 24.4	0 / 0
Clients: 20	3	64.69 ± 4.01	251.6 ± 28.9	650.9 ± 49.6	0 / 0
Clients: 50	3	73.20 ± 15.10	552.5 ± 102.7	1603.4 ± 429.3	0 / 0
$k = 0$	3	55.63 ± 2.59	68.9 ± 1.3	216.0 ± 32.0	0 / 0
$k = 1$	3	43.89 ± 1.87	91.2 ± 8.0	248.2 ± 5.9	0 / 0
$k = 4$	3	32.39 ± 2.26	139.9 ± 10.0	280.7 ± 28.5	0 / 0
Difficulty: 16 bits	6	40.38 ± 0.69	105.8 ± 4.9	261.9 ± 18.7	0 / 0
Difficulty: 18 bits	6	11.47 ± 1.17	385.1 ± 57.6	952.5 ± 139.9	0 / 0
Difficulty: 20 bits	6	2.94 ± 0.67	1428.1 ± 333.4	3346.4 ± 827.8	5 / 5
Block capacity: 1	3	27.79 ± 1.90	119.0 ± 6.4	360.3 ± 40.2	1 / 7
Block capacity: 10	6	41.63 ± 2.25	100.9 ± 7.1	244.1 ± 13.8	0 / 0
Block capacity: 50	6	37.29 ± 2.02	114.2 ± 8.0	291.9 ± 16.3	0 / 0
Block capacity: 100	6	37.72 ± 7.30	116.0 ± 25.5	303.7 ± 128.5	0 / 0
Replicas: 7 (5 clients)	6	31.13 ± 2.77	138.9 ± 12.9	333.6 ± 43.6	0 / 0

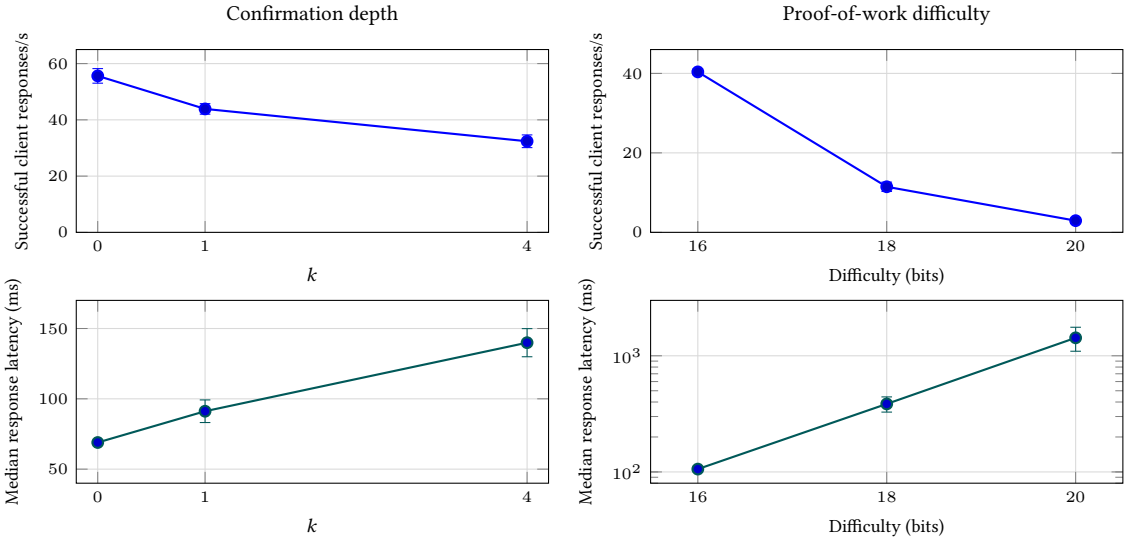


Figure 3: Blockchain performance sensitivity to confirmation depth and mining difficulty. Points show means and error bars one sample standard deviation; k settings use three runs and difficulty settings use six. The latency axis for difficulty is logarithmic.

Table 5: Blockchain S1–S6 safety scenario definitions and checks.

ID	Scenario	Primary check
S1	Valid and invalid blocks/transfers	Reject invalid proof of work, parent, height, size, signature or duplicate transfer; ensure later valid input can still be accepted.
S2	Duplicate and out-of-order delivery	Redeliver blocks and deliver a child before its parent; check idempotence and orphan attachment.
S3	Competing forks before finality	Deliver competing valid branches in different orders; check deterministic fork choice and convergence.
S4	Fork after payment execution	Challenge a branch containing an executed transfer; record chain choice and the no-rollback conflict.
S5	Byzantine miner behavior	Withhold blocks, disseminate invalid blocks or selectively send competing valid blocks within the fault bound.
S6	Shared-account conflicts	Issue concurrent transfers for the same account sequence; reconcile executed IDs, balances and replica digests.

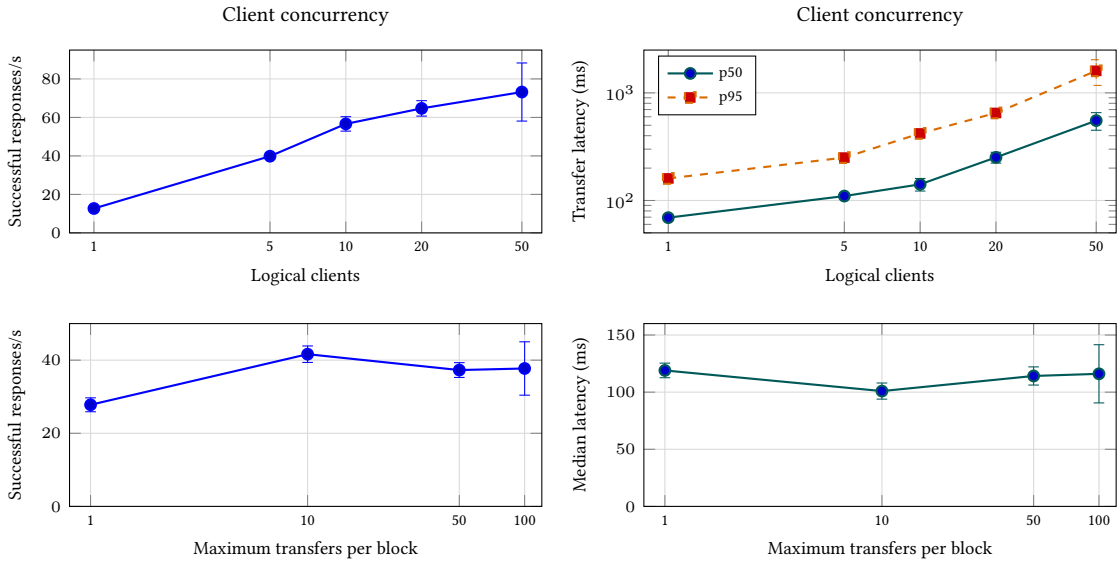


Figure 4: Blockchain client load and block-capacity sweeps. Points show means and error bars one sample standard deviation. Load points use three runs; block-capacity 1 uses three runs and capacities 10, 50 and 100 use six. Latency axes show means of the per-run percentiles.

Table 6: Blockchain outcome under verified 10-second node4 egress loss.

Workload	Run	Successful client replies	Same final digest	Supply
30 s	1	2169	4/4	Yes
30 s	2	1990	3/4	Yes
30 s	3	1917	3/4	Yes
90 s	1	5653	4/4	Yes
90 s	2	5961	4/4	Yes
90 s	3	5618	4/4	Yes

establish recovery during an idle period or identify which recovery mechanism caused convergence.

S6: Shared-account conflicts. Blockchain shared-account stress runs used four replicas, difficulty 16, $k = 2$, 100% shared transfers, 0 ms added delay and no replica CPU or memory cgroup limits. With two owners, two of three repetitions converged and one produced three distinct correct-replica digests. Supply was conserved in all three. With five owners, all three recorded repetitions converged and conserved supply. This sample is too small to infer that five owners are safer; it demonstrates that contention and run scheduling affect observed outcomes. Divergent-reply counts are lower bounds because replies arriving after a client accepts a quorum are not included.

6.3 Consensus-free BRB evaluation

6.3.1 Performance. The four-replica BRB client-count sweep used an 80% transfer mix, no shared-account transfers, and three 30-second runs per setting. Table 7 reports mean transfer throughput and the mean of each run's median latency. Throughput did not consistently grow beyond five clients: it remained around 320–360 transfers/s, while median latency increased from 8.3 ms at five clients to 85.0 ms at fifty. At fifty clients, balance reads timed out 228 times in total across the three repetitions. Transfer requests had no client-reported failures or timeouts in this sweep. The ten-client throughput has high run-to-run variation, which is retained in the standard deviation rather than excluded. Figure 5 shows the load and shared-account sweeps.

With five logical clients, increasing the replica count from four to seven reduced completed transfer throughput from 339.0 ± 8.9 to 157.8 ± 12.4 transfers/s; mean per-run median latency rose from 8.3 ± 0.7 to 24.6 ± 2.5 ms. The seven-replica runs had no transfer timeouts and two balance-read timeouts in total. This local comparison includes additional message fan-out and replica work, and is not a general scalability result. The operation-mix sweep measured 56,607 ± 3,694 balance reads/s for read-only clients. At a 50/50 mix, transfer throughput was 339.6 ± 10.9/s and balance throughput was 337.5 ± 14.2/s. With only transfers, throughput was 347.4 ± 28.3/s. Table 8 also reports per-operation p50 and p95 latency. Balance reads are answered locally; their high read-only rate should not be interpreted as BRB write capacity. The 50/50 runs recorded ten balance-read timeouts in total; the transfer-only runs had no client-reported transfer timeouts.

Client-reported transfer latency ends when the client receives $f+1$ matching replies. For Blockchain, it is not a direct measurement of block propagation, mining time, or inclusion-to-execution time.

Table 7: BRB load sweep with four replicas. Values are mean ± sample standard deviation across three runs. Latencies are means of the per-run transfer p50 and p95 values; timeout counts are summed across runs.

Clients	Transfers/s	p50 (ms)	p95 (ms)	Transfer timeouts	Balance timeouts
1	320.8 ± 6.1	2.05 ± 0.09	5.59 ± 0.88	0	0
5	339.0 ± 8.9	8.26 ± 0.67	45.52 ± 6.35	0	2
10	361.3 ± 52.2	16.97 ± 3.01	66.20 ± 7.75	0	6
20	329.0 ± 19.1	48.47 ± 10.76	101.88 ± 8.23	0	40
50	340.1 ± 15.2	85.02 ± 9.29	242.08 ± 34.49	0	228

Table 8: BRB operation-mix sweep with four replicas and five clients. Values are mean ± sample standard deviation across three runs. Throughput counts successful client responses per second; latency columns are means of per-run percentiles. A dash means that operation type was not issued.

Transfer mix	Balance reads			Transfers		
	Ops/s	p50 (ms)	p95 (ms)	Ops/s	p50 (ms)	p95 (ms)
0%	56607 ± 3694	0.082 ± 0.005	0.12 ± 0.02	–	–	–
50%	337.5 ± 14.2	0.52 ± 0.11	2.01 ± 0.21	339.6 ± 10.9	6.88 ± 0.71	45.39 ± 3.56
100%	–	–	–	347.4 ± 28.3	9.19 ± 0.28	42.42 ± 8.83

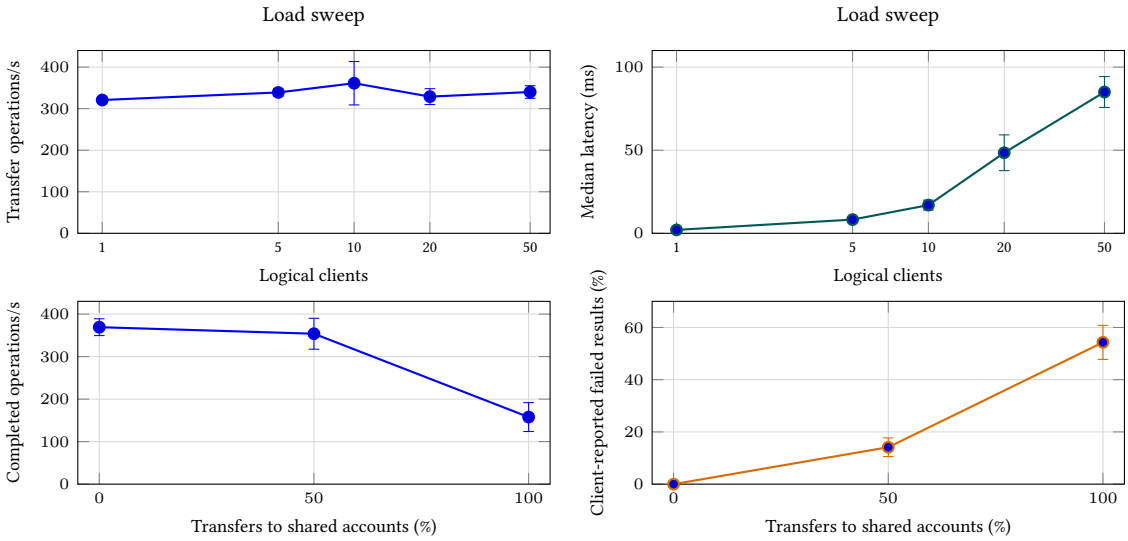


Figure 5: BRB performance and shared-account outcomes. Points show the mean across three runs and error bars show one sample standard deviation. The load plots use a logarithmic client axis. Correct-replica state divergence occurred in 0/3, 3/3, and 3/3 shared-account runs at 0%, 50%, and 100%, respectively.

We therefore interpret it as end-to-end request-completion latency. Percentiles are retained per run; percentiles from separate clients or runs are not averaged into a global percentile.

6.3.2 *Safety: S1–S8.*

S1–S4: Deterministic protocol and replication checks. Deterministic tests exercise BRB delivery after the READY quorum, duplicate and READY-before-ECHO messages, a single member's conflicting vote, and rejection of non-member votes. Replication tests verify that invalid transfers are ignored, duplicate deliveries execute once, and rejection does not prevent a later valid delivery. The complete Maven test suite passed.

S5: Selective-send Byzantine replica. Three process-level selective-send runs configured node4 as the sole Byzantine replica. Its action logs confirm that it sent SEND messages to only two peers and omitted its local ECHO. The correct replicas converged and conserved supply in every repetition, with no transfer failures or timeouts. In this client workload, each request is sent to all replicas, so the other correct origins also disseminated the same transfer; this result does not cover every selective Byzantine schedule.

S6: Shared-account conflicts. For BRB, the shared-account sweep varied the fraction of transfers directed to shared accounts while holding four replicas and five concurrent clients fixed. Completed-operation throughput was 369.2 ± 19.8, 353.8 ± 36.4, and 157.8 ± 34.0 operations/s at 0%, 50% and 100% shared use, respectively. At

Table 9: BRB safety scenarios and checks.

ID	Scenario and primary check
S1	Delivery waits for $2f + 1$ READY votes and occurs at most once.
S2	Duplicate READY messages and READY-before-ECHO ordering do not count a sender twice and still permit relay.
S3	Conflicting votes from one member cannot form a second quorum for the same instance.
S4	Votes from non-members do not count toward a quorum.
S5	One Byzantine replica selectively sends messages; check correct-replica safety and progress for this schedule.
S6	Concurrent transfers contend for a shared account; reconcile outcomes, balances and final digests.
S7	A replica crashes within the fault bound; surviving replicas continue and state is checked.
S8	A replica loses network egress temporarily; check behavior during the fault and convergence after restoration.

0% shared use, all three runs converged. At 50% and 100%, all three runs ended with divergent correct-replica snapshots. The client-reported failed-result fraction was $14.1 \pm 3.6\%$ and $54.3 \pm 6.5\%$, respectively. Each run's fraction is failed transfer results divided by all completed transfer results (successful plus failed); timeouts are reported separately and excluded from that denominator. The shared-use runs had 15 timeouts in total at 50% and 8 at 100% across the three repetitions. Every audited replica conserved total supply, and the audit records classified all shared transfer IDs as executed or conflicting. A client-reported failed result does not establish that no replica executed that transfer; divergent replicas can make different local decisions. These observations are consistent with per-instance reliable broadcast: it delivers each instance consistently, but imposes no total order across different transfers. Concurrent transfers can therefore contend for the same account sequence number and be accepted in different orders. Resolving this requires an ordering or arbitration mechanism beyond this variant.

S7: Replica crash. Three BRB crash runs stopped node4 ten seconds after client startup. The three remaining replicas continued to execute transfers and converged to matching full snapshots in every run. Transfer operations did not fail or time out; between one and three balance reads timed out per repetition.

S8: Temporary egress loss and restoration. Three further runs dropped all network egress from node4 for ten seconds and then restored it. The harness recorded both fault boundaries, and all replicas converged after restoration with conserved supply. No transfer failed or timed out in those runs; balance reads had zero to three timeouts per repetition.

6.4 PBFT evaluation

PBFT experiments used the supplied closed-loop client and Docker harness. The main configuration was four replicas ($n = 4$, $f = 1$), batch size 10, 50 ms batch timeout, 5 s primary suspicion timeout, checkpoint interval 100, no injected network delay, and no per-container CPU or memory limit. After adding quorum-certified tail catch-up, we reran the full P1–P6 performance matrix. It contains 61 runs: three repetitions per setting, except for four repetitions of the P5 egress interruption. The runs used 10,000 retained sequence numbers and a 10,000-entry fetch cap. P1 used a 20 s measurement interval, 3 s preparation and 10 s cooldown; P2–P4 and P6 used a 15 s interval, 3 s preparation and 5 s cooldown; P5 used a 20 s interval. Earlier pre-fix performance results are excluded from the tables and figures below. Performance latency is measured at the client until it receives matching replies; these are local single-host measurements. Docker container resource sampling ran once per second during all new runs.

6.4.1 Performance: P1–P6. Table 10 identifies the PBFT performance scenarios and summarizes the repeated evidence. Table 11 and Figure 6 show the P1 load sweep. The performance measurements include the signed decision-proof retention introduced for catch-up; the separate memory comparison below measures that change against the pre-fix implementation.

With one client, the closed-loop transfer rate was about 16/s; five concurrent clients raised it to about 499/s. Throughput then stayed near 558–585/s at 10–50 clients, while p95 transfer latency rose from 14.57 ms at five clients to 130.47 ms at fifty. This throughput plateau and rising tail latency are consistent with saturation on the shared host, but do not establish a stable maximum capacity. All 15 P1 runs ended with matching replica digests and conserved supply. Transfers had no timeouts; 21 balance reads timed out in total, all at 20 or 50 clients. Client-side reply timeouts therefore remain a high-load observation even though the replicas converged after the workload.

P2's seven-replica runs processed 201.21 ± 12.35 transfer replies/s at p95 91.67 ± 5.19 ms. The four-replica P1 ten-client point reached 558.40 ± 12.30/s at p95 32.12 ± 3.23 ms. This indicates lower throughput and higher latency with seven replicas on this host, although the run durations differ, so the result is indicative rather than a strictly matched scaling estimate. All seven replicas converged in all three P2 runs.

In P3, all four shared-account settings ended with matching replica digests and conserved supply in all repetitions. The mean number of successful and rejected shared transfers per run was 614 and 53 at 10%, 2,257 and 1,022 at 50%, and 3,237 and 3,242 at 100%. At zero shared use there were no shared-transfer attempts. These rejects are application conflicts caused by concurrent use of the same account sequence numbers, not replica divergence. The measured result separates deterministic ordering from transaction validity: PBFT gives replicas a common order, while the ledger rejects transfers that lose the account-sequence race.

P4's read-heavy mix averaged 461.19 ± 1.23 transfer replies/s at p95 20.84 ± 0.88 ms, 920.62 ± 4.72 history replies/s at p95 5.71 ± 0.26 ms, and 916.36 ± 8.09 balance replies/s at p95 5.71 ± 0.18 ms across three runs. Every run converged. These operation rates are consistent with the workload's 20/40/40 mix and should be interpreted separately by operation type.

For P6, batch capacities 1, 10, 50 and 100 reached respectively 248.04 ± 8.36, 500.49 ± 27.91, 137.72 ± 2.43 and 139.04 ± 0.80 transfer replies/s. Their p95 latencies were 63.01 ± 1.67, 32.48 ± 3.71, 77.94 ± 3.22 and 76.61 ± 1.83 ms. Capacity 10 performed best in this workload. Capacities 50 and 100 had lower observed throughput and higher latency; the harness did not record batch occupancy, so we cannot attribute this difference to under-filled batches alone. With batch size 10, timeout settings of 10, 50, 200 and 1000 ms produced 490.19 ± 40.60, 511.52 ± 23.75, 496.48 ± 11.01 and 495.94 ± 14.69 transfer replies/s, with p95 latency between 31.27 and 37.95 ms. The differences show no monotonic benefit from increasing the timeout.

Table 10: PBFT P1–P6 scenarios and post-fix local results.

ID	Scenario	Variation and primary measurements	Evidence
P1	Client load	1, 5, 10, 20 and 50 logical clients; transfer throughput and client latency.	Three runs per load; all final replicas converged after the state-transfer fix.
P2	Replica-count scaling	Seven replicas ($f = 2$), with ten clients.	Three runs; all replicas converged in each run. Compared with the four-replica P1 ten-client point.
P3	Shared-account contention	0%, 10%, 50% and 100% shared transfers with four concurrent clients.	Three runs per setting; all final replica digests matched. Shared-transfer rejects rose with contention.
P4	Read-heavy mix	20% transfers, 40% history and 40% balance requests; ten clients.	Three runs; all converged. Reads had higher throughput and lower latency than transfers.
P5	Fault recovery	Four-replica primary egress loss for 5 s, then restoration.	Four runs; all replicas converged after restoration. Primary crashes are covered by the safety experiment.
P6	Batching	Batch capacity 1, 10, 50 and 100; timeout 10, 50, 200 and 1000 ms.	Three runs per setting with corrected history limits; every run converged.

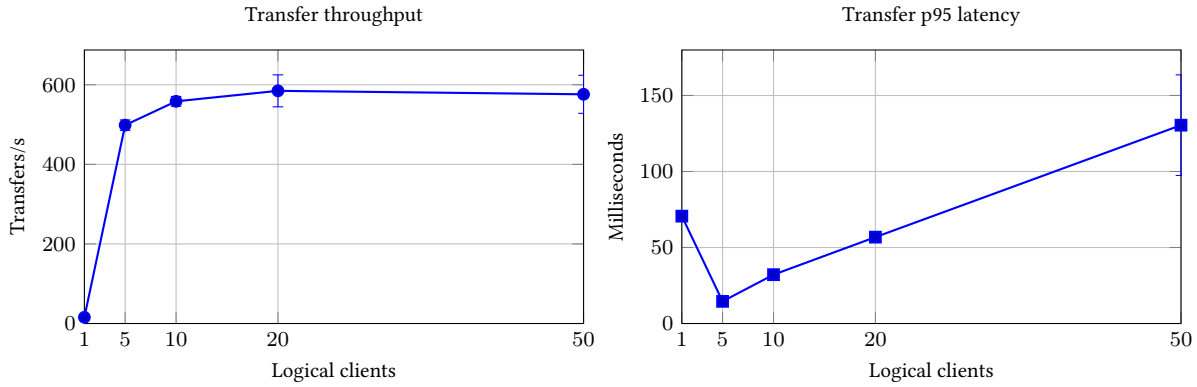


Figure 6: PBFT P1 client-load results. Markers show the mean across three runs and error bars show one standard deviation.

Table 11: PBFT P1 transfer results after the tail catch-up fix. Throughput and p95 latency are mean ± standard deviation across three runs.

Clients	Runs	Transfers/s	p95 (ms)	Converged
1	3	15.63 ± 0.06	70.64 ± 0.43	3/3
5	3	498.54 ± 13.13	14.57 ± 0.45	3/3
10	3	558.40 ± 12.30	32.12 ± 3.23	3/3
20	3	584.75 ± 40.18	56.82 ± 3.31	3/3
50	3	576.08 ± 47.82	130.47 ± 33.10	3/3

In P5, the primary's egress was dropped for 5 s and then restored in four runs. The transfer rate averaged 407.45 ± 15.11/s at p95 17.22 ± 1.12 ms during the 20 s workload. The harness recorded both fault boundaries in all runs; all four replicas converged with conserved supply after restoration. This is evidence for recovery from the tested temporary isolation, not a guarantee for arbitrary network partitions.

We measured memory against the pre-fix source at revision `c9dfa2e` using Docker's once-per-second container statistics, taking each replica container's largest sampled usage within a run and averaging those per-container peaks. These numbers include JVM heap, native memory and container overhead; they are not heap measurements. The three paired workloads show small positive mean changes, from about 3 to 12 MiB per replica. The paired run spread is larger than the mean change for P1 and P2; P6 shows a more consistent increase, but three runs are insufficient to claim a precise general memory cost. This comparison measures the

Table 12: Sampled per-replica peak Docker memory before and after the PBFT tail catch-up fix. Each value is mean ± standard deviation across three runs; Δ is computed from paired repetitions.

Workload	Pre-fix (MiB)	Post-fix (MiB)	Paired Δ (MiB)
P1: 50 clients, $n = 4$	314.88 ± 8.98	321.32 ± 33.42	+6.44 ± 41.91
P2: 10 clients, $n = 7$	301.26 ± 13.56	304.53 ± 3.86	+3.27 ± 10.33
P6: batch size 1, $n = 4$	290.10 ± 5.23	301.66 ± 0.55	+11.56 ± 5.28

additional retention of signed decision proofs against the pre-fix implementation under these workloads and durations.

6.4.2 Safety and recovery. The primary equivocation profile sent two correctly signed, conflicting pre-prepare values for the same view and sequence number, then withheld its commit for that sequence. This is a targeted test profile, not a general Byzantine adversary. Table 13 summarizes the process-level outcomes.

The equivocation results support a limited safety conclusion: under the tested signed equivocation schedule, we observed no two correct replicas deciding different values for the attacked sequence. The four-replica runs recovered through view change. The initial seven-replica runs exposed a recovery gap: a replica in view change could catch up to a stable checkpoint but remained behind on later committed entries while the other replicas continued making progress. We fixed this by retaining signed decision certificates and fetching the uncheckpointed tail; the receiver verifies the primary signature and a quorum of distinct commit signatures before executing fetched entries. In four post-fix seven-replica reruns, all six correct replicas converged to identical audited ledger digests within the 30 s cooldown, with supply conserved. This validates recovery for the tested profile and schedule, not arbitrary Byzantine behavior or forged certificates. The P1–P6 performance matrix has now been rerun after this recovery change, with a sampled-memory comparison in Table 12. The high-load P1 results also depended on state-transfer retention: before that separate fix, a lagging replica requested batches already pruned from peers' logs and remained behind. Holding the transfer target fixed and increasing the retained history and fetch cap to 10,000 eliminated unservable or over-cap fetches in the 15 corrected P1 runs. This is a bounded recovery improvement; a replica that falls farther behind than the configured retained history still needs a snapshot-based transfer mechanism.

6.5 Cross-protocol comparison

6.5.1 Latency overhead. In terms of latency, BRB is the best-performing protocol on overall tested configurations. Although it comes with a trade-off - the lack of consistency - in the sense that when accounts are shared, concurrent transfers with the same account sequence number cause correct replicas to accept operations in conflicting orders, leading to divergent ledger states. PBFT, by contrast, incurs more latency overhead because it guarantees a total order of operations, ensuring ledger convergence. The Blockchain primitive showed significantly higher latency across all configurations because of its proof-of-work computation. For instance, in a 20-client configuration, Blockchain presented 5 times more latency overhead than the BRB solution, considering p50.

6.5.2 Throughput. PBFT achieved the highest overall transfer throughput. This is because of its batching technique: it gathers a set of operations before proposing them as a single operation to the membership, enabling much more effective total ordering. BRB, on the other hand, processes every transfer independently, with per-instance state management and per-sender vote tracking for every transaction, creating heavy per-operation messaging overhead that limits scaling. Blockchain is indeed the slowest protocol because replicas spend nearly one hundred per cent of their 1-vCPU time computing SHA-256 nonces, severely throttling transaction inclusion.

6.5.3 Shared-accounts consistency. This is where our protocols differ most. Because they have different philosophies, even when they agree on specifications, they can deviate from the desired behavior. The BRB variant has a ledger divergence because it cannot fully order operations, allowing concurrent broadcasts by two different clients of the same account to be accepted at different instances on distinct replicas. The Blockchain Proof-of-Work Blockchain uses a probabilistic approach, introducing a parameter k that determines when to confirm an operation. Still, this parameter is only a stability heuristic and provides no formal guarantees of total transaction ordering, so it can still suffer from concurrent shared-account transfer ledger divergence. PBFT is strongest in that respect, since it establishes a total order of operations across all replicas by gathering quorums of messages that confirm replicas are ready to decide and have seen the operation in question. In that sense, all replicas' ledgers converge, all agreeing on a concrete serialization of concurrent shared-account transactions.

6.5.4 Safety. Under the targeted Byzantine profiles aligned with the most critical attack vectors for each protocol, none deviated from the specification! Blockchain showed that correct replicas converged to matching chain digests and conserved total supply across all repetitions, even with invalid block dissemination, block withholding, and selective dissemination. PBFT succeeded in tolerating equivocation attacks, and BRB tolerated a selective-sending replica. The only inevitable downside is the need for at least three times as many replicas as faulty replicas [7]; otherwise, safety is compromised and all guarantees discarded.

6.5.5 Interpretation. Blockchain replication offers a simple longest-chain ordering rule and probabilistic stability controlled by mining power and k. Its practical costs in these experiments are mining work and confirmation delay. The design does not provide deterministic finality: a deep reorganization can conflict with irreversible payment execution. The BRB-based consensus-free variant also has no total order across instances; our shared-account runs showed conflicting sequence outcomes and divergent correct-replica ledgers

Table 13: PBFT safety and recovery observations. A deadline miss is reported as a liveness observation for the tested schedule, not as proof of permanent failure.

Scenario	Configuration and repetitions	Observed result
Primary equivocation	$n = 4$, $f = 1$; three runs.	Correct replicas changed view and converged in all runs. No conflicting final decision or ledger state was observed; transfers resumed after the no-op decision at the equivocated sequence.
Primary equivocation	$n = 7$, $f = 2$; four post-fix targeted runs, one Byzantine primary; 30 s cooldown.	All six correct replicas converged to identical audited ledger digests in all four runs; supply was conserved. The initial implementation had left one correct replica behind in three runs. The fix fetches the uncheckpointed tail with a quorum of signed commit proofs.
Primary crash	$n = 4$, $f = 1$; three runs, primary stopped after 5 s.	All three surviving replicas converged; no transfer timeouts were recorded.
Primary egress loss	$n = 4$, $f = 1$; three 5 s isolation and restoration runs.	All replicas converged after connectivity returned; audited snapshots matched and supply was conserved in all runs.
Shared-account contention	$n = 4$, $f = 1$; three audited 100% shared runs.	All four replicas matched their final snapshots in each run and conserved supply; same-account sequence conflicts were reported by the application.

despite supply conservation. PBFT is intended to establish a deterministic total order, and our local PBFT runs converged under the tested four- and seven-replica equivocation schedules and primary-crash schedule after quorum-certified tail catch-up was added. High-load runs also required a larger retained-history window for recovery. These protocol-specific runs do not form a matched three-way comparison: workloads, container limits, durations and settings differ. All measurements are local and preliminary.

7 CONCLUSION

We study three designs for Byzantine fault-tolerant payment replication over Babel 2: consensus-free replication using BRB, state machine replication using PBFT, and a Proof-of-Work blockchain. This draft reports local Docker experiments for all three alternatives, while leaving the matched three-way comparison open. The measurements show that proof-of-work difficulty and confirmation depth affect throughput and latency. BRB achieved high local read throughput, but shared-account runs exposed divergent correct-replica states because independent broadcasts have no global order. Irreversible payment execution is also incompatible with arbitrary deep-chain reorganization unless the system adds a finality mechanism or the application supports rollback. The results are preliminary and local. PBFT throughput rose with client concurrency before flattening near the tested host's capacity; equivocation and crash experiments exposed recovery behavior, and the seven-replica catch-up gap was fixed and rerun successfully in four trials. The PBFT results use different resource limits and parameter settings from the other alternatives, so matched comparisons remain future work.

REFERENCES

[1] Gabriel Bracha. 1987. Asynchronous Byzantine Agreement Protocols. *Information and Computation* 75, 2 (1987), 130–143.

[2] Christian Cachin, Rachid Guerraoui, and Luís Rodrigues. 2011. *Introduction to Reliable and Secure Distributed Programming* (2nd ed.). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-15260-3

[3] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI)*, Vol. 99. 173–186.

[4] Daniel Collins, Rachid Guerraoui, Jovan Komatovic, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, Yvonne-Anne Pignolet, Dragos-Adrian Seredinschi, Andrei Tonkikh, and Athanasios Xygkis. 2020. Online Payments by Merely Broadcasting Messages. In *Proceedings of the 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 26–38.

[5] Pedro Fouto, Pedro Ákos Costa, Nuno Preguiça, and João Leitão. 2022. Babel: A Framework for Developing Performant and Dependable Distributed Protocols. In *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*. IEEE Computer Society, Los Alamitos, CA, USA, 146–155. https://doi.org/10.1109/SRDS55811.2022.00022

[6] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovič, and Dragos-Adrian Seredinschi. 2019. The Consensus Number of a Cryptocurrency. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*. ACM, 307–316.

[7] Dahlia Malkhi and Michael Reiter. 1998. Byzantine Quorum Systems. *Distributed Computing* 11, 4 (1998), 203–213. https://doi.org/10.1007/s004460050050

[8] Satoshi Nakamoto. 2009. Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf.